Upgrade utils

Upgrade utils is a library that contains helper functions to facilitate the writing of upgrade scripts. This library, used by Odoo for the upgrade scripts of standard modules, provides reliability and helps speed up the upgrade process:

- The helper functions help make sure the data is consistent in the database.
- It takes care of indirect references of the updated records.
- Allows calling functions and avoid writing code, saving time and reducing development risks.
- Helpers allow to focus on what is important for the upgrade and not think of details.

Installation

Clone the Upgrade utils repository locally and start `odoo` with the `src` directory prepended to the `--upgrade-path` option.

```
$ ./odoo-bin --upgrade-path=/path/to/upgrade-util/src,/path/to/other/upgrade/script
```

On platforms where you do not manage Odoo yourself, you can install this library via `pip` :

```
$ python3 -m pip install git+https://github.com/odoo/upgrade-util@master
```

On Odoo.sh it is recommended to add it to the *requirements.txt* of the custom repository. For this, add the following line inside the file:

```
odoo_upgrade @ git+https://github.com/odoo/upgrade-util@master
```

Using upgrade utils

Once installed, the following packages are available for the upgrade scripts:

- **`odoo.upgrade.util`** : the helper itself.

odoo docs

```
from odoo.upgrade import util


def migrate(cr, version):
    # Rest of the script
```

Now, the helper functions are available to be called through `util`.

Util functions

Upgrade utils provides many useful functions to ease the upgrade process. Here, we describe some of the most useful ones. Refer to the **util folder** for the comprehensive declaration of helper functions.

> **Note**
>
> The `cr` parameter in util functions always refers to the database cursor. Pass the one received as a parameter in `migrate()`. Not all functions need this parameter.

Modules

Utility functions for module-level operations.

In most cases module operations (rename, merge, remove, ...) should be performed in a `base` script. The reason is that once the `base` module is upgraded, all the information regarding modules should be already set in the DB for the upgrade process to function correctly. The command line option `--pre-upgrade-scripts` (available from Odoo 16) allows to run upgrade scripts *before* loading base. This is the recommended way to perform module operations after a major upgrade.

modules_installed(*cr*, **modules*) [source]

Return whether *all* given modules are installed.

Parameters:
modules (*list* (*str*)) – names of the modules to check
Return type:
bool

odoo docs

upgrade, or for installation, even if they are not yet fully installed.

module_installed(*cr*, *module*) [source]

Return whether a module is installed.

Parameters:
module (*str*) – name of the module to check
Return type:
bool

See `modules_installed()` .

uninstall_module(*cr*, *module*) [source]

Uninstall and remove all records owned by a module.

Parameters:
module (*str*) – name of the module to uninstall

uninstall_theme(*cr*, *theme*, *base_theme=None*) [source]

Uninstall a theme module and remove it from websites.

Parameters:
theme (*str*) – name of the theme module to uninstall

base_theme (*str* *or* *None*) – if not `None` , unload first this base theme

> **Warning**
>
> This function can only be used in `post-` scripts of `website` module as it relies on the ORM.

See `remove_theme()` and `uninstall_module()` .

remove_module(*cr*, *module*) [source]

Completely remove a module.

This operation is equivalent to uninstall and removal of *all* references to the module - no trace of it is left in the database.

odoo docs

> **Warning**
>
> Since this function removes *all* data associated to the module. Ensure to reassign records before calling this function.

remove_theme(*cr*, *theme*, *base_theme=None*) [source]

Uninstall a theme module.

> **Warning**
>
> This function can only be used in `post-` scripts.

See **remove_module()** and **uninstall_theme()** .

rename_module(*cr*, *old*, *new*) [source]

Rename a module and all references to it.

Parameters:
old (*str*) – current name of the module to rename
new (*str*) – new name of the module to rename

merge_module(*cr*, *old*, *into*, *update_dependers=True*) [source]

Merge a module into another.

This function moves all references and records from the source module to the destination module.

> **Warning**
>
> This functions does not remove any record, but it removes xml_ids from the source module with a conflicting name in the destination module.

Parameters:
old (*str*) – name of the module to be merged
into (*str*) – name of the module to merge into

odoo docs

force_install_module(cr, module, if_installed=None, reason='it has been explicitly asked for') [source]

Force the ORM to install a module.

Parameters:
module (*str*) – name of the module to install
if_installed (*list* (*str*) *or* *None*) – only force the install when these modules are already installed
Return str:
the *new* state of the module

force_upgrade_of_fresh_module(cr, module, init=True) [source]

Force the execution of upgrade scripts for a module that is being installed.

Standard Odoo doesn't run upgrade scripts when a module is being installed. This makes sense because, technically speaking, the module is not being upgraded. Still, it happens that a (new) module needs to perform some operations for it to be correctly installed, like grabbing data from another module. This is common when a module is functionally *split* into several ones.

Parameters:
module (*str*) – name of the module for which to force the execution of upgrade scripts
init (*bool*) – whether to set the module to be in *init* mode

Being in init mode has the side effect of not respecting noupdate flags, in XML file nor in `ir_model_data`.

move_model(cr, model, from_module, to_module, move_data=False, keep=())
[source]

Move a model from one module to another.

Parameters:
model (*str*) – name of the model to move
from_module (*str*) – name of the module where the model is originally defined
to_module (*str*) – name of the destination module where the model is to be moved
move_data (*bool*) – whether to also update `ir_model_data` for records of the model
keep (*list* (*str*)) – list of XML ids to keep - not move

odoo docs

Models

Utility functions for modifying models.

Model operations are best done in `pre-` script of the involved modules.

remove_model(*cr*, *model*, *drop_table=True*, *ignore_m2m=()*) [source]

Remove a model and its references from the database.

Some required indirect references to the model are replaced by the *unknown model* - an empty model that serves as placeholder for gone models.

Parameters:
model (*str*) – name of the model to remove

drop_table (*bool*) – whether to drop the table of this model

ignore_m2m (*list* *(str)* *or* *str*) – list of m2m tables to ignore - not removed, use `"*"` to ignore (keep) all m2m tables

delete_model(*cr*, *model*, *drop_table=True*, *ignore_m2m=()*) [source]

Remove a model and its references from the database.

Some required indirect references to the model are replaced by the *unknown model* - an empty model that serves as placeholder for gone models.

Parameters:
model (*str*) – name of the model to remove

drop_table (*bool*) – whether to drop the table of this model

ignore_m2m (*list* *(str)* *or* *str*) – list of m2m tables to ignore - not removed, use `"*"` to ignore (keep) all m2m tables

rename_model(*cr*, *old*, *new*, *rename_table=True*, *ignored_m2ms='ALL_BEFORE_18_1'*) [source]

Rename a model.

Updates all references to the model name in the DB.

If table rename is requested, from saas~18.1+, m2m table are updated too, unless ignored. In older versions, m2m tables are skipped unless an empty list is passed.

odoo docs

rename_table (*bool*) – whether to also rename the table of the model

ignored_m2ms – m2m tables to skip. Defaults to `"ALL_BEFORE_18_1"`, which skips all in Odoo 18 or below, none in saa~18.1+. For all versions, if the value is not the default, skip only the specified m2m tables.

merge_model(*cr*, *source*, *target*, *drop_table=True*, *fields_mapping=None*, *ignore_m2m=()*) [source]

Merge a model into another one.

This function moves all the references from `source` model into `target` model and *removes* `source` model and its references. By default, only the fields with the same name in both models are moved from source to target, optionally a mapping with differently named fields can be provided.

> **Warning**
>
> This function does not move the records from `source` model to `target` model.

Parameters:

source (*str*) – name of the model to be merged

target (*str*) – name of the destination model to merge into

drop_table (*bool*) – whether to drop the table of the source model

fields_mapping (*dict* *or* *None*) – mapping of field names from source to target model, when `None` only fields with same name are moved

ignore_m2m (*list* *(str)* *or* *str*) – list of m2m tables ignored to remove from source model.

remove_inherit_from_model(*cr*, *model*, *inherit*, *keep=()*, *skip_inherit=()*, *with_inherit_parents=True*, *skip_update_references=()*) [source]

Remove `inherit` from `model`.

This function removes all fields inherited via `inherit` from a `model` and all its descendant models. All fields from the inherit, including those from its parent models are also removed, unless `with_inherit_parents` mode is unset. In such case only fields from `inherit` are removed, excluding fields in its parents. Fields listed in `keep` are never removed. If some descendants from `model` are listed in `skip_inherit` they will keep the fields from `inherit`.

odoo docs

keep (*tuple* (*str*)) – tuple with field names to keep

skip_inherit (*tuple* (*str*)) – list of descendant models of `model` to not process

with_inherit_parents (*boolean*) – if unset, remove fields coming from `inherit` only, keeping all fields from its parents

skip_update_references (*tuple* (*str*) *or "*"*) – field names whose references should not be updated upon deletion. Use `"*"` for all fields.

Fields

Utility functions for modifying model fields.

Field operations are best done in `pre-` script of the involved modules. In some cases a preliminary operation could be done in pre and finished in post. A common example is to remove a field in pre- keeping its column, later used in post when the column is finally dropped.

remove_field(*cr*, *model*, *fieldname*, *cascade=False*, *drop_column=True*, *skip_inherit=()*, *keep_as_attachments=False*, *update_references=True*) [source]

Remove a field and its references from the database.

This function also removes the field from inheriting models, unless exceptions are specified in `skip_inherit`. When the field is stored we can choose to not drop the column.

Parameters:

model (*str*) – model name of the field to remove

fieldname (*str*) – name of the field to remove

cascade (*bool*) – whether the field column(s) are removed in `CASCADE` mode

drop_column (*bool*) – whether the field's column is dropped

skip_inherit (*list* (*str*) *or* *str*) – list of inheriting models to skip the removal of the field, use `"*"` to skip all

keep_as_attachments (*bool*) – for binary fields, whether the data should be kept as attachments

update_references (*bool*) – whether to update all references. If `False`, don't update dashboard, filters, export paths, domains, and relations for related fields.

make_field_non_stored(*cr*, *model*, *field*, *selectable=False*) [source]

Convert field to non-stored.

odoo docs

selectable (*bool*) – whether the field is still selectable, if True custom `ir.filters` are not updated

move_field_to_module(*cr*, *model*, *fieldname*, *old_module*, *new_module*, *skip_inherit=()*) [source]

Move a field from one module to another.

This functions updates all references to a specific field, switching from the source module to the destination module. It avoid data removal after the registry is fully loaded. The field in inheriting models are also moved unless skipped.

Parameters:

model (*str*) – name of the owner model of the field to move

fieldname (*str*) – name of the field to move

old_module (*str*) – source module name from which the field is moved

new_module (*str*) – target module name into which the field is moved

skip_inherit (*list* (*str*) *or* *str*) – list of inheriting models for which the field is not to be moved, use `"*"` to skip all

rename_field(*cr*, *model*, *old*, *new*, *update_references=True*, *domain_adapter=None*, *skip_inherit=()*) [source]

Rename a field and its references from `old` to `new` on the given `model`.

The field is updated in all inheriting models, except for models specified in `skip_inherit`.

This functions also updates references, indirect or direct ones, including filters, server actions, related fields, emails, dashboards, domains, and many more. See **update_field_usage()**

For the update of domains a special adapter function can be used. The default adapter just replaces `old` by `new` in each domain leaf. Refer to **adapt_domains()** for information about domain adapters.

Parameters:

model (*str*) – model name of the field to rename

old (*str*) – current name of the field to rename

new (*str*) – new name of the field to rename

use `"*"` to skip all

invert_boolean_field(*cr*, *model*, *old*, *new*, *skip_inherit=()*) [source]

Rename a boolean field and invert its value.

convert_m2o_field_to_m2m(*cr*, *model*, *field*, *new_name=None*, *m2m_table=None*, *col1=None*, *col2=None*) [source]

Convert a Many2one field to a Many2many.

It creates the relation table and fill it.

Parameters:

- **model** (*str*) – model name of the field to convert
- **field** (*str*) – current name of the field to convert
- **new_name** (*str*) – new name of the field to convert. If not provide, it's `field` with a trailing "s".
- **m2m_table** (*str*) – name of the relation table. Automatically generated if not provided.
- **col1** (*str*) – name of the column referencing `model`.
- **col2** (*str*) – name of the column referencing the target of `field`.

m2o2m2m(*cr*, *model*, *field*, *new_name=None*, *m2m_table=None*, *col1=None*, *col2=None*) [source]

Convert a Many2one field to a Many2many.

It creates the relation table and fill it.

Parameters:

- **model** (*str*) – model name of the field to convert
- **field** (*str*) – current name of the field to convert
- **new_name** (*str*) – new name of the field to convert. If not provide, it's `field` with a trailing "s".
- **m2m_table** (*str*) – name of the relation table. Automatically generated if not provided.
- **col1** (*str*) – name of the column referencing `model`.
- **col2** (*str*) – name of the column referencing the target of `field`.

change_field_selection_values(*cr*, *model*, *field*, *mapping*, *skip_inherit=()*) [source]

are also updated.

Parameters:
model (*str*) – model name of the selection field to update

field (*str*) – name of the selection field to update

mapping (*dict*) – selection values to update, key values are replaced by their corresponding values in the mapping

skip_inherit (*list* *(str)* *or* *str*) – list of inheriting models to skip in the update of the selection values, use `"*"` to skip all

`update_field_usage(cr, model, old, new, domain_adapter=None, skip_inherit=())` `[source]`

Replace all references to the field `old` by `new` in different places.

Search in:

- ir_filters
- ir_exports_line
- ir_act_server
- mail_alias
- ir_ui_view_custom (dashboard)
- sign_item_type
- sign_item
- domains (using `domain_adapter`)
- related fields

This function can be used to replace the usage of a field by another. Domains are updated using the `domain_adapter` . By default the domain adapter just replaces `old` by `new` in domain leaves. See `adapt_domains()` for more information about domain adapters.

Parameters:
model (*str*) – model name of the field

old (*str*) – source name of the field to replace

new (*str*) – target name of the field to set

domain_adapter (*function*) – adapter to use for domains, see `adapt_domains()`

skip_inherit (*list* *(str)* *or* *str*) – models to skip when renaming the field in inheriting models, use `"*"` to skip all

odoo docs

remove_view(*cr*, *xml_id=None*, *view_id=None*, *silent=False*, *key=None*) [source]

Remove a view and all its descendants.

This function recursively deletes the given view and its inherited views, as long as they are part of a module. It will fail as soon as a custom view exists anywhere in the hierarchy. It also removes multi-website COWed views.

Parameters:

xml_id (*str*) – optional, the xml_id of the view to remove

view_id (*int*) – optional, the ID of the view to remove

silent (*bool*) – whether to show in the logs disabled custom views

key (*str* *or* *None*) – key used to detect multi-website COWed views, if `None` then set to `xml_id` if provided, otherwise set to the xml_id referencing the view with ID `view_id` if any

> **Warning**
>
> Either `xml_id` or `view_id` must be set. Specifying both will raise an error.

edit_view(*cr*, *xmlid=None*, *view_id=None*, *skip_if_not_noupdate=True*, *active='auto'*) [source]

Context manager to edit a view's arch.

This function returns a context manager that may yield a parsed arch of a view as an **etree Element**. Any changes done in the returned object will be written back to the database upon exit of the context manager, updating also the translated versions of the arch. Since the function may not yield, use **`skippable_cm()`** to avoid errors.

```
with util.skippable_cm(), util.edit_view(cr, "xml.id") as arch:
    arch.attrib["string"] = "My Form"
```

To select the target view to edit use either `xmlid` or `view_id`, not both.

When the view is identified by `view_id`, the arch is always yielded if the view exists, with disregard to any `noupdate` flag it may have associated. When `xmlid` is set, if the view `noupdate` flag is `False` then the arch will not be yielded *unless* `skip_if_not_noupdate` is set to `False`. If `noupdate` is `True`, the view will be yielded for edit.

odoo docs

> **Note**
>
> If `active` is “auto” (default value), the view will be activated if selected via `xmlid` and left untouched if selected via `view_id`.

Parameters:
xmlid (*str*) – optional, xml_id of the view edit

view_id (*int*) – optional, ID of the view to edit

skip_if_not_noupdate (*bool*) – whether to force the edit of views requested via `xmlid` parameter even if they are flagged as `noupdate=True`, ignored if `view_id` is set

active (*bool* *or* *None* *or* *"auto"*) – active flag value to set. Unchanged when `None`.

Returns:
a context manager that yields the parsed arch, upon exit the context manager writes back the changes.

`remove_record(cr, name)` [source]

Remove a record and its references corresponding to the given **xml_id**.

Parameters:
name (*str*) – record xml_id, under the format `module.name`

`is_changed(cr, xmlid, interval='1 minute')` [source]

Return whether a record was changed.

This function checks if a record was changed before the current upgrade start time. See *upgrade-util/src/base/0.0.0/pre-00-upgrade-start.py*

This utility will return a false positive on xmlids of records that match the following conditions:

- Have been updated in an upgrade preceding the current one
- Have not been updated in the current upgrade

If the `xmlid` doesn’t exist in the DB this function returns `None`.

Parameters:
xmlid (*str*) – `xmlid` of the record to check

odoo docs

if_unchanged(*cr*, *xmlid*, *callback*, *interval='1 minute'*, ***kwargs*) [source]

Run `callback` if a record is unchanged.

This function will run a `callback` when the referred record is unchanged. The `xmlid` and any extra parameter, but not `interval`, will be passed to the `callback`. In case the record was changed it will be marked as `noupdate=True`. See also **`is_changed()`** and **`force_noupdate()`**.

This function is useful to take an action *only* when a record hasn't been updated, a common example is to force an update from XML even if the record was `noupdate=True`

```
util.if_unchanged(cr, "mymodule.myrecord", util.update_record_from_xml)
```

Parameters:

xmlid (*str*) – xml_id of the record to check

callback (*function*) – callback to execute in case the record was *not* changed, all extra parameters to this function are passed to the callback

interval (*str*) – interval after `create_date` on which a record is considered as _changed_, see **`is_changed()`**

rename_xmlid(*cr*, *old*, *new*, *noupdate=None*, *on_collision='fail'*) [source]

Rename an **external identifier** (`xml_id`) of a record.

A rename cannot happen when the target name already exists on the database. In such cases there are two options to control how this function behaves:

- `fail` : raise an exception and prevent renaming
- `merge` : rename the external identifier, remove the old one, and *replace* references. See **`replace_record_references_batch()`**

> **Note**
>
> This function does not remove records, it only updates xml_ids.

Parameters:

odoo docs

on_collision (*str*) – how to proceed if the xml_id already exists, the options are `merge` or `fail` (default)

Returns:
the ID of the record referenced by the *new* xml_id, `None` when the record doesn't exist

Return type:
int or None

ref(*cr*, *xmlid*) [source]

Return the id corresponding to the given xml_id.

Parameters:
xml_id (*str*) – record xml_id, under the format `module.name`

Returns:
ID of the referenced record, `None` if not found.

Return type:
int or None

force_noupdate(*cr*, *xmlid*, *noupdate=True*, *warn=False*) [source]

Update the `noupdate` flag of a record.

Parameters:
xmlid (*str*) – record xml_id, under the format `module.name`

noupdate (*bool*) – value to set on the `noupdate` flag

warn – whether to output a warning in the logs when the flag was switched from `True` to `False`

ensure_xmlid_match_record(*cr*, *xmlid*, *model*, *values*) [source]

Ensure an xml_id references a record with specific values.

This function ensures the record pointed by an xml_id matches the values for the fields specified in the `values` parameter. When the xmlid exist but it points to a record that doesn't match the values, the xmlid is updated to point to a record that matches the values if one is found. If the xmlid doesn't exist, it is created with the found record. When no matching record is found, nothing is done. In all cases the referenced record, after a possible update, of the xml_id is returned.

Parameters:
xmlid (*str*) – record xml_id, under the format `module.name`

odoo docs

Example

```
values = {"id": 123}
values = {"name": "INV/2024/0001", "company_id": 1}
```

Returns:
the ID of the matched record, `None` if no record found
Return type:
int or None

Tip

This function is useful when migrating in-database records into a custom module, to create the xml_ids before the module is updated and avoid duplication.

update_record_from_xml(*cr*, *xmlid*, *reset_write_metadata=True*, *force_create=AUTO*, *from_module=None*, *reset_translations=()*, *ensure_references=False*, *fields=None*) [source]

Update a record based on its definition in the Data Files.

This function ignores the `noupdate` flag on the record. It searches in all XML files from the manifest of the module in the xmlid, or the `from_module` parameter if set, for a matching definition. When found, it forces the ORM to update the record as in the specs in the data file.

Optionally this function can reset the translations of some fields.

Parameters:
xmlid (*str*) – record xml_id, under the format `module.name`

reset_write_metadata (*bool*) – whether to update the `write_date` of the record

force_create (*bool*) – whether the record is created if it does not exist. `True` by default, unless `fields` is not None.

from_module (*str*) – name of the module from which to update the record, necessary only when the specs are in a different module than the one in the xml_id

reset_translations (*set* (*str*)) – field names whose translations get reset

odoo docs

other fields will be ignored. When set, record won't be created if missing.

> **Warning**
>
> This functions uses the ORM, therefore it can only be used after **all** models referenced in the data specs of the record are already **loaded**. In practice this means that this function should be used in `post-` or `end-` scripts.

> **Note**
>
> The standard behavior of ORM is to create the record if it doesn't exist, including its xml_id. That will happen on this function as well, unless `force_create` is set to `False`.

delete_unused(*cr*, **xmlids*, ***kwargs*) [source]

Remove unused records.

This function will remove records pointed by `xmlids` only if they are not referenced from any table. For hierarchical records (like product categories), it verifies if the children marked as cascade removal are also not referenced. In which case the record and its children are all removed.

> **Note**
>
> The records that cannot be removed are set as `noupdate=True`.

Parameters:

xmlids (*list* *(str)*) – list of xml_ids to check for removal

deactivate (*bool*) – whether to deactivate records that cannot be removed because they are referenced, `False` by default

keep_xmlids (*bool*) – whether to keep the xml_ids of records that cannot be removed. By default `True` for versions up to 18.0, `False` from `saas~18.1` on.

include_m2m (*list* *(str)* *or* *str*) – list of m2m tables to include in the search. `"*"` for all.

Returns:

list of ids of removed records, if any

Return type:

list (int)

odoo docs

This functions replaces *all* references, direct or indirect to records of `model_src` by the corresponding records in the mapping. If the target model of the mapping is not the same as the source one, then `model_dst` parameter must be set.

Parameters:

id_mapping (*dict* *(int, int)*) – mapping of IDs to replace, key value is replaced by the mapped value

model_src (*str*) – name of the source model of the records to replace

model_dst (*str*) – name of the target model of the records to replace, if `None` the target is assumed the same as the source

replace_xmlid (*bool*) – whether to replace the references in xml_ids pointing to the source records

ignores (*list* *(str)*) – list of **table** names to skip when updating the referenced values

Paream str parent_field:

when the target and source model are the same, and the model table has `parent_path` column, this field will be used to update it.

`replace_in_all_jsonb_values(cr, table, column, old, new, extra_filter=None)` [source]

Replace values in JSONB columns.

This function replaces `old` by `new` in JSONB values. It is useful for replacing values in *all* translations of translated fields.

Parameters:

table (*str*) – table name where the values are to be replaced

column (*str*) – column name where the values are to be replaced

old (*str*) – original value to replace, can be a simple term (str) or a Postgres regular expression wrapped by `PGRegexp`

new (*str*) – new value to set, can be a simple term or a expression using `<number>` notation to refer to *captured groups* if `old` is a regexp expression

extra_filter (*str*) – extra `WHERE` compatible clause to filter the values to update, must use the `t` alias for the table, it can also include `{parallel_filter}` to execute the query in parallel, see `explode_execute()`

ORM

odoo docs

cases totally different alternatives to the ORM's own functions are provided. The functions on this module work along the ORM of *all* supported versions.

env(*cr*) [source]

Create a new environment.

> **Warning**
>
> This function does *not* empty the cache maintained on the cursor for superuser with an empty environment. A call to `invalidate_cache` may be necessary every time data is modified directly in the database.

Returns:
the new environment
Return type:
Environment

recompute_fields(*cr*, *model*, *fields*, *ids=None*, *logger=<Logger odoo.upgrade.util.orm (WARNING)>*, *chunk_size=256*, *strategy='auto'*, *query=None*) [source]

Recompute field values.

This function will recompute fields of a model restricted to a set of records - or all. The re-computation is not done on all records at the same time. It is split in batches (chunks). This avoids performance issues, and, in the worse cases, failures due to `MemoryError`. After each chunk is processed the data is sent back to the database according to one of the following strategies:

- *flush*: use the `flush` method of the ORM
- *commit*: `commit` the cursor - also flush
- *auto*: pick the best alternative between the two above given the number of records to compute and the presence of tracked fields.

The *commit* strategy is less prone to cause a `MemoryError` for a huge volume of data.

Parameters:
model (*str*) – name of the model to recompute

fields (*list* (*str*)) – list of the name of the fields to recompute

odoo docs

chunk_size (*int*) – number of records per chunk - used to split the processing

strategy (*str*) – strategy used to process the re-computation

query (*str*) – query to get the IDs of records to recompute, it is an error to set both `ids` and `query`. Note that the processing will always happen in ascending order. If that is unwanted, you must use `ids` instead.

class **iter_browse**(*model*, **args*, ***kw*) [source]

Iterate over recordsets.

The `callable` object returned by this class can be used as an iterator that loads records by chunks (into a `recordset`). After each chunk is exhausted their data is sent back to the database - flushed or committed - and a new chunk is loaded.

This class allows to run code like:

```
for record in env['my.model'].browse(ids):
    record.field1 = val

env['my.model'].browse(ids)._compute_field2()
env['my.model'].create(values)
```

in a performant way while also avoiding `MemoryError`, even when `ids` or `values` have millions of entries. The alternative using this class would be:

Example

```
MyModel = util.env(cr)['my.model']
for record in util.iter_browse(MyModel, ids):
    record.field1 = val

util.iter_browse(MyModel, ids)._compute_field2()
util.iter_browse(MyModel, ids).create(values)
```

Parameters:

model (**`odoo.model.Model`**) – the model to iterate

ids (*list* (*int*)) – list of IDs of the records to iterate

odoo docs

logger (`logging.Logger`) – logger used to report the progress, by default `_logger`

strategy (*str*) – whether to `flush` or `commit` on each chunk, default is `flush`

Returns:
the object returned by this class can be used to iterate, or call any model method, safely on millions of records.

See also `env()`

create(*values*, ***kw*) [source]

Create records.

An alternative to the default `create` method of the ORM that is safe to use to create millions of records.

Parameters:
values (*list* (*dict*)) – list of values of the records to create

multi (*bool*) – whether to use the multi version of `create` , by default is `True` from Odoo 12 and above

normalize_domain(*domain*) [source]

Return a normalized version of the domain.

The normalized version have all implicit '&' operators explicit. One property of normalized domain expressions is that they can be easily combined together as if they were single domain components.

adapt_domains(*cr*, *model*, *old*, *new*, *adapter=None*, *skip_inherit=()*, *force_adapt=False*) [source]

Replace `old` by `new` in domains using `model` and inheriting models.

`adapter` is a callback function to adapt leaves. Adapter functions must take three arguments and return a **domain** that substitutes the original leaf. The arguments are:

- `leaf` : a domain leaf which is a `tuple` of the form `(left, op, right)`
- `in_or` : a boolean, when `True` it means the leaf is part of an OR (`"|"`) domain, otherwise it is part of an AND (`"&"`) domain
- `negated` : a boolean, when `True` it means that the leaf is negated (`"!"`)

```
    left, op, right = leaf
    ok, ko = (1, 2) if not negated else (2, 1)
    if op == "="
        return [(left, "=", ok)]
    elif op == "!=":
        return [(left, "=", ko)]
    return [leaf]
```

`adapter` is called only on leaves that use the `old` field of `model` as **last** part of the `left` part of leaves, unless `force_adapt` is `True`. In the latter case the adapter is called if the field appears anywhere in the path, useful for relational fields only.

The domains returned by an adapter do not need to have the `old` field replaced by `new` in the `left` part of the input leaf. The replace will be done anyway to the whole domain returned by the adapter. The usual purpose of the `adapter` is to modify the operator and the `right` part of the input leaf. When `adapter` is not set only the replacement takes place.

Example

When replacing `"field1"` by `"field2"`, the following happens:

- `("foo.bar.baz.field1", "=", 1)` gets adapted *only* if the record pointed to by `foo.bar.baz` is of the requested `model`.
- `("foo.field1.baz", "=", 1)` is *not* adapted *even* if `foo` points to `model`, unless `force_adapt` is `True`, because `field1` is not the last part of `left` in this leaf.

Note

This function will replace domains in all *standard* domain fields. Including filters, dashboards, and standard fields known to represent a domain.

Parameters:

model (*str*) – name of the model for which to adapt the domains

old (*str*) – name of the field to be adapted

new (*str*) – name of the field that should replace `old`

adapter (*function*) – adapter for leaves

odoo docs

SQL

Utility functions for interacting with PostgreSQL.

class **PGRegexp** [source]

Wrapper for semantic meaning of parameters: this string is a Postgres regular expression.

See `replace_in_all_jsonb_values()`

class **SQLStr** [source]

Wrapper for semantic meaning of parameters: this string is a valid SQL snippet.

See `format_query()`

parallel_execute(*cr*, *queries*, *logger=<Logger odoo.upgrade.util.pg (WARNING)>*) [source]

Execute queries in parallel.

Example

```
util.parallel_execute(cr, [util.format_query(cr, "REINDEX TABLE {}", t) for t
```

Tip

If looking to speedup a single query, see `explode_execute()`.

Parameters:
queries (*list* (*str*)) – list of queries to execute concurrently
logger (*Logger*) – logger used to report the progress
Returns:
the sum of `cr.rowcount` for each query run
Return type:
int

Warning

odoo docs

taken into account.
- As a side effect, the cursor will be committed.

> **Note**
>
> If a concurrency issue occurs, the *failing* queries will be retried sequentially.

format_query(*cr*, *query*, **args*, ***kwargs*) [source]

Safely format a query.

The `str` arguments to this function are assumed to be SQL identifiers. They are wrapped in double quotes before being expanded using **`str.format()`**. Any other psycopg2.sql.Composable are also allowed. This includes **`ColumnList`**, see also **`get_columns()`**

> **Example**
>
> ```
> >>> util.format_query(cr, "SELECT {0} FROM {table}", "id", table="res_users")
> SELECT "id" FROM "res_users"
> ```

Parameters:
query (*str*) – query to format, can use brackets `{}` as in **`str.format()`**

explode_execute(*cr*, *query*, *table*, *alias=None*, *bucket_size=10000*, *logger=<Logger odoo.upgrade.util.pg (WARNING)>*) [source]

Execute a query in parallel.

The query is split by buckets of ids, then processed in parallel by workers. If the query does not include the special `{parallel_filter}` value, it is added to the last `WHERE` clause, possibly also adding it if none found. When the query already has the filter nothing is done. The filter always expands to the splitting strategy. The split is done into buckets where no more than `bucket_size` IDs are updated on each individual query.

> **Example**

odoo docs

```
    UPDATE res_users u
       SET active = False
     WHERE (u.login LIKE 'dummy' OR u.login = 'bob')
       AND {parallel_filter}
    ''',
    table="res_users"
    alias="u",
)
```

Parameters:
query (*str*) – the query to execute.
table (*str*) – name of the *main* table of the query, used to split the processing
alias (*str*) – alias used for the main table in the query
bucket_size (*int*) – size of the buckets of ids to split the processing
logger (`logging.Logger`) – logger used to report the progress
Returns:
the sum of `cr.rowcount` for each query run
Return type:
int

Warning

It's up to the caller to ensure the queries do not update the same records in different buckets. It is advised to never use this function for `DELETE` queries on tables with self references due to the potential `ON DELETE` effects. For more details see `parallel_execute()`.

`column_exists(cr, table, column)` [source]

Return whether a column exists.

Parameters:
table (*str*) – table to check
column (*str*) – column to check
Return type:
bool

odoo docs

Parameters:
table (*str*) – table to check
column (*str*) – column to check
Return type:
SQL type of the column

create_column(*cr*, *table*, *column*, *definition*, ***kwargs*) [source]

Create a column.

This function will create the column only if it *doesn't* exist. It will log an error if the existing column has different type. If `fk_table` is set, it will ensure the foreign key is setup, updating if necessary, with the right `on_delete_action` if any is set.

Parameters:
table (*str*) – table of the new column
column (*str*) – name of the new column
definition (*str*) – column type of the new column
default (*bool*) – default value to set on the new column
fk_table (*bool*) – if the new column if a foreign key, name of the foreign table
on_delete_action (*str*) – `ON DELETE` clause, default `NO ACTION`, only valid if the column is a foreign key.
Returns:
whether the column was created
Return type:
bool

copy_column(*cr*, *table*, *column*, *new_name=AUTO*) [source]

Copy a column.

This function copies a column if it exists. It raises an error otherwise.

Parameters:
table (*str*) – table of the column
column (*str*) – name of the column
new_name (*str*) – name of the new column, if not passed the original column name will be used with suffix `_upg_copy`.
Returns:

odoo docs

alter_column_type(*cr*, *table*, *column*, *type*, *using=None*, *where=None*, *logger=<Logger odoo.upgrade.util.pg (WARNING)>*) [source]

Alter the type of a column.

Do it efficiently using a temporary column and parallel UPDATE queries.

Parameters:

table (*str*) – name of the affected table.

column (*str*) – name of the column to alter type.

type (*str*) – new type of the column.

using (*str*) – SQL expression that define how to convert the value to the new type. The `{0}` placeholder will be replaced by the column name.

where (*str*) – used to restrict the values converted via `using`.

logger (`logging.Logger`) – logger used to report the progress.

remove_constraint(*cr*, *table*, *name*, *cascade=False*, *warn=True*) [source]

Remove a table constraint.

This function removes the constraint `name` from `table`. It also removes records from `ir_model_constraint` and its xml_ids. It logs not found constraints.

> **Note**
>
> If there is no constraint `name`, this function will attempt to remove `{table}_{name}`, the latter is the default name the ORM uses for constraints created from `_sql_constraints`.

Parameters:

table (*str*) – table from where to remove the constraint

name (*str*) – name of the constraint to remove

cascade (*bool*) – cascade the constraint removal

warn (*bool*) – use warning level when logging not found constraints, otherwise use info level

Returns:

whether the constraint was removed

Return type:

odoo docs

Encapsulate a list of elements that represent column names.

The resulting object can be rendered as string with leading/trailing comma or an alias.

Parameters:
list (*list* (*str*)) – list of column names

quoted (*list* (*str*)) – list of quoted column names, it must correspond with the `list_` parameter

Example

```
>>> columns = ColumnList(["id", "field_Yx"], ["id", '"field_Yx"'])
```

```
>>> list(columns)
['id', '"field_Yx"']
```

```
>>> columns.using(alias="t").as_string(cr._cnx)
'"t"."id", "t"."field_Yx"'
```

```
>>> columns.using(leading_comma=True).as_string(cr._cnx)
', "id", "field_Yx"'
```

```
>>> util.format_query(cr, "SELECT {} t.name FROM table t", columns.using(alia
'SELECT "t"."id", "t"."field_Yx", t.name FROM table t'
```

```
>>> columns = ColumnList.from_unquoted(cr, ["foo", "BAR"])
```

```
>>> list(columns)
['"foo"', '"BAR"']
```

odoo docs

> **Note**
>
> This class is better used via `get_columns()`

classmethod **from_unquoted**(*cr*, *list_*) [source]

Build a ColumnList from a list of column names that may need quoting.

Parameters:
list (*list* *(str)*) – list of unquoted column names

using(*leading_comma=KEEP_CURRENT*, *trailing_comma=KEEP_CURRENT*, *alias=KEEP_CURRENT*) [source]

Set up parameters to render this list as a string.

Parameters:
leading_comma (*bool*) – whether to render a leading comma in front of this list

trailing_comma (*bool*) – whether to render a trailing comma

alias (*str* *or* *None*) – alias of the table of the columns, no alias is added if set to `None`

Returns:
a copy of the list with the parameters set

Return type:
`ColumnList`

iter_unquoted() [source]

Iterate over the raw column names, non quoted.

This is useful if the quoting is done outside this object. Also to get access to raw column names as in Postgres catalog.

Returns:
an iterator for the raw column names

get_columns(*cr*, *table*, *ignore=('id',)*) [source]

Return a list of columns in a table.

odoo docs

Returns:
a list of column names present in the table
Return type:
`ColumnList`

`get_common_columns(cr, table1, table2, ignore=('id',))` [source]

Return a list of columns present in both tables.

Parameters:
table1 (*str*) – first table name whose columns are retrieved

table2 (*str*) – second table name whose columns are retrieved

ignore (*list* *(str)*) – list of column names to ignore in the returning list
Returns:
a list of column names present in both tables
Return type:
`ColumnList`

`rename_table(cr, old_table, new_table, remove_constraints=True)` [source]

Rename a table.

This function renames the table `old_table` into `new_table`, as well as its primary key (and sequence), indexes, and foreign keys.

Parameters:
old_table (*str*) – name of the table to rename

new_table (*str*) – new name of the table
Para bool remove_constraints:
whether to remove the table constraints

`create_m2m(cr, m2m, fk1, fk2, col1=None, col2=None)` [source]

Ensure a m2m table exists or is created.

This function creates the table associated to a m2m field. If the table already exists, `fixup_m2m()` is run on it. The table name can be generated automatically, applying the same logic of the ORM. In order to do so, use the value "auto" for the `m2m` parameter.

Parameters:

odoo docs

col1 (*str*) – column referencing `fk1`, defaults to `"{fk1}_id"`

col2 (*str*) – column referencing `fk2`, defaults to `"{fk2}_id"`

Returns:
the name of the table just created/fixed-up

Return type:
str

class **named_cursor**(*cr*, *itersize=None*) [source]

Server side cursor.

This class wraps a psycopg2 server-side cursor. It adds convenient methods like `dictfetchmany` and `dictfetchall`. It should be used as a context manager.

Server-side cursors are useful to fetch huge amounts of data from the DB by chunks while at the same time keep using the main upgrade cursor.

Parameters:
itersize (*int*) – determines the number of rows fetched from PG at once.

bulk_update_table(*cr*, *table*, *columns*, *mapping*, *key_col='id'*) [source]

Update table based on mapping.

Each `mapping` entry defines the new values for the specified `columns` for the row(s) whose `key_col` value matches the key.

Example

```
# single column update
util.bulk_update_table(cr, "res_users", "active", {42: False, 27: True})

# multi-column update
util.bulk_update_table(
    cr,
    "res_users",
    ["active", "password"],
    {
        "admin": [True, "1234"],
        "demo": [True, "5678"],
    },
```

odoo docs

Parameters:
table (*str*) – table to update.

columns (*str* | *list*(*str*)) – columns spec for the update. It could be a single column name or a list of column names. The `mapping` must match the spec.

mapping (*dict*) – values to set, which must match the spec in `columns`, following the **same** order

key_col (*str*) – column used as key to get the values from `mapping` during the update.

> **Warning**
>
> The values in the mapping will be casted to the type of the target column. This function is designed to update scalar values, avoid setting arrays or json data via the mapping.

class **query_ids**(*cr*, *query*, *itersize=None*) [source]

Iterator over ids returned by a query.

This allows iteration over a potentially huge number of ids without exhausting memory.

Parameters:
query (*str*) – the query that returns the ids. It can be DML, e.g. `UPDATE table WHERE ... RETURNING id`.

itersize (*int*) – determines the number of rows fetched from PG at once, see `named_cursor()`.

Misc

Miscellaneous standalone functions.

version_gte(*version*) [source]

Return whether currently running Odoo version is greater or equal to `version`.

This function is useful for conditional execution in an upgrade script that applies to multiple versions, e.g. `0.0.0` scripts.

Parameters:
version (*str*) – Odoo version, must follow the format `[saas~]X.Y` where `X` is the major Odoo version, `saas~` is necessary only when `Y` is nonzero
Return type:
bool

odoo docs

See also `version_gte()`

> **Note**
>
> The bounds are inclusive.

Parameters:
a (*str*) – Odoo version, lower bound
b (*str*) – Odoo version, upper bound
Return type:
bool

`expand_braces(s)` [source]

Expand braces in the input.

> **Example**
>
> ```
> >>> util.expand_braces("a_{this,that}_b")
> ['a_this_b', 'a_that_b']
> ```

Parameters:
s (*str*) – string to expand, must contain precisely one pair of braces.
Returns:
expanded input

`import_script(path, name=None)` [source]

Import an upgrade script.

This function allows to import functions from other upgrade scripts into the current one.

> **Example**
>
> In *mymodule/15.0.1.0/pre-migrate.py*
>
> ```
> def my_util(cr):
>     # do stuff
> ```

odoo docs

```
script = util.import_script("mymodule/15.0.1.0/pre-migrate.py")

def migrate(cr, version):
    script.my_util(cr)  # reuse the function
```

This function returns a Python `module`.

```
>>> util.import_script("base/0.0.0/end-moved0.py", name="my-moved0")
<module 'my-moved0' from '/home/odoo/src/upgrade-util/src/base/0.0.0/end-move
```

Parameters:
path (*str*) –
relative path to the script to import in the form `$module/$version/$script-name`

> **Note**
>
> The script must be available in the upgrade path.

name (*str* *or* *None*) – name to assign to the returned module, take the name from the imported file if `None`

Returns:
a module created from the imported upgrade script

`skippable_cm()` [source]

Return a context manager to allow another context manager to not yield.

See `edit_view()` for an example usage.

`chunks(iterable, size, fmt=None)` [source]

Split `iterable` into chunks of `size` and wrap each chunk using `fmt` function.

This function is useful for splitting huge input data into smaller chunks that can be processed independently.

> **Example**

```
'abc def ghi jkl m'
```

Parameters:
iterable (*iterable*) – iterable object to split

size (*int*) – chunk size

fmt (*function*) – function to apply to each chunk, when `None` is passed `fmt` becomes `"".join` if `iterable` is a string, otherwise `iter`

Returns:
a generator that iterates over the result of `fmt` applied to each chunk

Testing upgrade scripts

There are two main classes used testing during the upgrade.

- **`UpgradeCase`** for testing upgrade scripts,
- **`UpgradeCase`** for testing invariants across versions.

Subclasses must implement:

- For `UpgradeCase`:
  - `prepare` method: prepare data before upgrade,
  - `check` method: check data was correctly upgraded.
- For `IntegrityCase`:
  - `invariant` method: compute an invariant to check.

Put your test classes in a `tests` Python module (folder) in any of the folders containing the upgrade scripts of your modules. The script containing your tests must have a `test_` prefix. The `tests` module must contain an `__init__.py` file to be detected by Odoo.

Example directory structure:

```
myupgrades/
└── mymodule1/
    ├── 18.0.1.1.2/
    │   └── pre-myupgrade.py
    └── tests/
```

odoo docs

> **Note**
>
> The tests in the example above will be loaded only if `mymodule1` is being **upgraded.**

Running Upgrade Tests

After receiving an upgraded database with all standard Odoo modules already upgraded to their target version, you can test the upgrade of custom modules by following a three-step process:

1. Prepare the test data

```
$ ~/odoo/$version/odoo-bin -d DB --test-tags=$prepare_test_tag \
  --upgrade-path=~/upgrade-util/src,~/myupgrades \
  --addons=~/odoo/$version/addons,~/enterprise/$version,~/mymodules --stop
```

2. Upgrade the modules

```
$ ~/odoo/$version/odoo-bin -d DB -u mymodule1,mymodule2 \
  --upgrade-path=~/upgrade-util/src,~/myupgrades \
  --addons=~/odoo/$version/addons,~/enterprise/$version,~/mymodules --stop
```

3. Check the upgraded data

```
$ ~/odoo/$version/odoo-bin -d DB --test-tags=$check_test_tag \
  --upgrade-path=~/upgrade-util/src,~/myupgrades \
  --addons=~/odoo/$version/addons,~/enterprise/$version,~/mymodules --stop
```

The example above assumes that `$version` is the target version of your upgrade (e.g. `18.0`), `DB` is the name of your database, and `mymodule1,mymodule2` are the modules you want to upgrade. The directory structure assumes that `~/odoo/$version` and `~/enterprise/$version` contain the Community and Enterprise source code for the target Odoo version, respectively. `~/mymodules` contains the code of your custom modules (`mymodule1`, ...), `~/myupgrades` contains your custom upgrade scripts, and `~/upgrade-util` contains the **upgrade utils** repo.

odoo docs

$prepare_test_tag	upgrade.test_prepare	integrity_case.test_prepare
$check_test_tag	upgrade.test_check	integrity_test.test_check

> **Note**
>
> `upgrade.test_prepare` also runs `IntegrityCase` tests, so you can prepare data for both `UpgradeCase` and `IntegrityCase` tests with only this tag.

> **Warning**
>
> Do **not** run any `prepare` method of an `UpgradeCase` before sending your database for a **production** upgrade to **upgrade.odoo.com**. Doing so may risk your upgrade being blocked and marked as failed.

API documentation

parametrize(*argvalues*) [source]

Parametrize a test function.

Decorator for upgrade test functions to parametrize and generate multiple tests from it. The new test functions are injected in the containing class. Inspired by the **parameterized** package.

> **Example**
>
> ```
> @parametrize([(1, 2), (2, 4), (-1, -2), (0, 0)])
> def test_double(self, input, expected):
>     self.assertEqual(input * 2, expected)
> ```
>
> This will generate four test methods: `test_double__0`, `test_double__1`, `test_double__2`, and `test_double__3`, each with different argument values.

Parameters:

odoo docs

change_version(version_str) [source]

Class decorator to specify the version on which a test is relevant.

Using `@change_version(version)` indicates:

- `test_prepare` will only run if the current Odoo version is in the range `[next_major_version-1, version)`.
- `test_check` will only run if the current Odoo version is in the range `[version, next_major_version)`.

`next_major_version` is the next major version after `version`, e.g. for `saas~17.2` it is `18.0`.

> **Note**
>
> Do not use this decorator if your upgrade is in the same major version. Otherwise, your tests will not run.

class **UpgradeCase**(*methodName='runTest'*) [source]

Test case to verify the upgrade flow.

Override:

- `prepare` to set up data,
- `check` to assert expectations after the upgrade.

The ORM can be used in these methods to perform the functional flow under test. The return value of `prepare` is persisted and passed as an argument to `check`. It must be JSON-serializable.

> **Note**
>
> Since `prepare` injects or modifies data, this type of test is intended **only for development**. Use it to test upgrade scripts while developing them. **Do not** run these tests for a production upgrade. To verify that upgrades preserved important invariants in production, use `IntegrityCase` tests instead.

> **Example**

odoo docs

```python
class DeactivateBobUsers(UpgradeCase):

    def prepare(self):
        u = self.env["res.users"].create({"login": "bob", "name": "Bob"})
        return u.id  # will be passed to check

    def check(self, uid):  # uid is the value returned by prepare
        self.env.cr.execute(
            "SELECT * FROM res_users WHERE id=%s AND NOT active", [uid]
        )
        self.assertEqual(self.env.cr.rowcount, 1)
```

class **IntegrityCase(*methodName='runTest'*)** [source]

Test case for validating upgrade invariants.

Override:

- `invariant` to return a JSON-serializable value representing the invariant to check.

The `invariant` method is called both before and after the upgrade, and the results are compared. If there is any difference the test fails.

Example

```python
from odoo.upgrade.testing import IntegrityCase


class NoNewUsers(IntegrityCase):
    def invariant(self):
        return self.env["res.users"].search_count([])
```

Get Help